



## L6: Spring Cleaning

### Stages:

Stage 1 (6 pts)

Stage 2 (5 pts)

Stage 3 (8 pts)

Stage 4 (5 pts)

Task 5 (0 pts)

[Starting code](#)

# L6: Shared Memory and Synchronization

## L6: Spring Cleaning

Despite the date of the announcement, students of the Faculty of Mathematical Linguistics at Skaryszew Polytechnic, aware of the problem, took the keyboard cleaning action in the laboratory rooms initiated by the student union very seriously. They crowdedly and eagerly supported this initiative and on Holy Saturday, instead of an Easter basket, they gathered on site with brushes, spatulas, and a set of disinfectants (student version).

The number of willing students exceeded the organizers' wildest expectations, and they began to fear that instead of effective work, even greater chaos would ensue. Therefore, they turned to their long-term partner, the MiNI faculty, to develop a system managing the cleaning process and handling unforeseen accidents.

The system is designed to oversee cleaning in a single room. The room contains  $m$  keyboards, each of which has  $k$  keys. To avoid crowding and properly allocate resources, it was assumed that no more than `KEYBOARD_CAP` students can be at one keyboard at the same time.

Help the students finish cleaning the faculty before Easter!

### Stages:

Stage 1 (6 pts)

The program takes three arguments: `KEYBOARD_CAP`  $\leq n \leq 20$  – the number of students cleaning the room – and  $1 \leq m \leq 5$  and  $5 \leq k \leq \text{KEYBOARD\_CAP}$  – as described before. Create  $n$  child processes (students). Each of them opens  $m$  **named semaphores** `/sop-sem-1`,  $\dots$ , `/sop-sem- $m$` . If a given semaphore does not exist, it should be created and initialized with the value `KEYBOARD_CAP`.

After opening the semaphore, the student process in a loop of 10 iterations chooses a random keyboard and then attempts to approach it, waiting if there is no space – use the appropriate semaphore. Then it prints `Student <PID>: cleaning keyboard <i>` (where  $i$  is the number of the drawn keyboard), waits 300ms, and releases the semaphore. After completing the loop, the process finishes.

The main process waits for all child processes to finish, then **removes** all created semaphores, prints `Cleaning finished!` and finishes.

**Immediately after the program starts**, remove the mentioned semaphores if they exist.

*Hint:* You can ignore the `ENOENT` error code in a specific function.

## Stage 2 (5 pts)

Before you model the keyboards in the program, restrict access to them before initialization. Before creating child processes, create a block of anonymous shared memory, which (for now) should store one **barrier shared between processes**. Create this barrier immediately after creating the shared memory so that students have access to it. The barrier should wait for a total of  $n + 1$  processes. Students, before creating semaphores, wait on the barrier. The main process, after creating the child processes, sleeps for 500ms and then waits on the barrier.

*Hint:* Creating a shared barrier is analogous to creating a shared mutex (instead of `pthread_mutexattr_t` use `pthread_barrierattr_t`).

## Stage 3 (8 pts)

Model the keyboard cleaning process in the system. Keyboards are represented by one common block of shared memory, which should be mapped from a **named memory object** named `SHARED_MEM_NAME`. The block should contain  $mk$  numbers of type `double`, one for each key in the keyboards. Perform the creation and mapping of the memory object in the main process **after creating the student processes**. Then initialize the shared memory by filling all its fields with ones (1.0). Make sure that no child process uses the contents of

the shared memory before initialization – use the barrier from the previous stage.

Student processes open and map the memory object `SHARED_MEM_NAME`, and then, after drawing a keyboard, they draw a key number and proceed to cleaning: the process should still wait 300ms, and then divide the value of the corresponding field by 3. Add information about the drawn key number to the message from Stage 1: `Student <PID>: cleaning keyboard <i>, key <j>.`

Protect student processes against simultaneous cleaning of the same key – use a separate mutex for each key. Place the mutexes, together with the barrier, in the shared memory from the previous stage.

After all student processes finish, the main process **prints the state** of all keyboards (use the provided `print_keyboards_state` function for this) and **removes** the shared memory object.

## Stage 4 (5 pts)

Undoubtedly, cleaning a keyboard is an exhausting activity, quickly draining all the cleaner's strength. This time, students perform the cleaning process in an infinite loop. However, during cleaning, a student has a 1% chance of falling from exhaustion. Simulate this by printing `Student <PID>: I have no more strength!`, releasing the semaphore, and calling the `abort()` function just before updating the dirt value on the key.

When a student attempts to clean a key that another student could not finish cleaning, they print `Student <PID>: someone is lying here, help!!!`, and then announce it to all students using a shared flag (place it and the mutex protecting it in the memory block next to other mutexes and the barrier). Students, concerned by such a situation, finish cleaning and run out of the room in panic (i.e., they break the loop and finish).

## Task 5 (0 pts)

**Congratulations!** Now it's your turn to clean the keyboard at your workstation.

## Starting code

- [Makefile](#)
- [sop-keys.c](#)



[Download as zip](#)